Introduction to Applied Cryptography – LDACS1310
Exercises Session 5

Questions

Question 1: Greatest Common Divisor

The Greatest Common Divisor of two integers a and b is the largest positive integer d such that d is a divisor of both a and b without a remainder.

1. The GCD can be naively computed by enumerating every possible divisor of a and b and keeping the biggest one. Implement the function `gcd_naive` in `e1.py`. What is the complexity of `gcd_naive` in term of $n = min(a, b)$?
2. A more efficient algorithm to compute the GCD is the Euclidean algorithm. The principle underlying it is to subtract the smaller number from the larger one until the smaller reaches zero. It is based on the principle that, if g divides a and b, with $a > b$, then it also divides $a - b$. For example, $\gcd(15, 10) = \gcd(10, 15 - 10) = 5$. Implement the Euclidean algorithm in the function `gcd_euclidean_sub` (`e1.py`). What is the complexity of `gcd_euclidean_sub` in term of $n = min(a, b)$?
3. Often $a - b$ is still larger than b. Indeed, for example with $a = 15$ and $b = 5$, we obtain $a - b = 15 - 5 = 10 > b = 5$. We need to subtract at least $\lfloor \frac{a}{b} \rfloor$ times to obtain $a - \lfloor \frac{a}{b} \rfloor b < b$. The equation $a - \lfloor \frac{a}{b} \rfloor b$ can be computed with $a \mod b$. Implement this modification of the Euclidean algorithm in the function `gcd_euclidean_mod` (`e1.py`).
4. The Extended Euclidean algorithm computes the gcd and the coefficients of Bézout's identity which are integers x and y such that $ax + by = \gcd(a, b)$. Implement the function `gcd_extended_euclidean` (`e1.py`) that also return x and y.

In the following, we will consider p prime and $x \in \mathbb{Z}_p$.

Question 2: Inverse

In $\mathbb{Z}_p$, an element x has a unique inverse (denoted x^{-1}) as long as $\gcd(x, p) = 1$. An inverse element of x is an element a where $a \cdot x \equiv 1 \pmod{p}$. To find this, we can compute a and y that solves this equation:

$$a \cdot x + p \cdot y = 1.$$

Once you get a, the inverse of x in $\mathbb{Z}_p$ is simply $a \pmod{p}$. Use the function `gcd_extended_euclidean` (`e1.py`) to implement `get_inverse` (`e2.py`) that returns the inverse of an element x in $\in \mathbb{Z}_p$.

Question 3: Square and multiply

Let $n \in \mathbb{Z}$. The purpose of this exercise is to compare two methods used to compute $x^n \pmod{p}$.

Note: To compute the exponentiation when $n < 0$, you can use the algorithm you implemented to compute the inverse as:

$$\forall i \in \mathbb{N}, x^{-i} \pmod{p} = (x^{-1})^n \pmod{p}.$$

1. First, implement the naive exponentiation `naive_exp` (`e2.py`), which consists in multiplying x by itself n times.
2. A more efficient way to compute the exponentiation is by using the following fact:

$$x^n \pmod p = \begin{cases} x(x^2)^{\frac{n-1}{2}} \pmod p & \text{if } n \text{ is odd,} \\ (x^2)^{\frac{n}{2}} \pmod p & \text{if } n \text{ is even.} \end{cases}$$

   This method is called square and multiply. Implement this function in the method `exp` in `e2.py`.
   After each step, you can run your code to test if the values are correct and to get the time comparison of these two methods.
3. In practice, we choose a p prime of the form $2kq + 1$ with q prime and k the minimal number such that $2kq + 1$ is prime. The goal is to work in a group $\mathbb{G}$ of size q that is generated by an element $g \in \mathbb{Z}_p$. To find an element $x \in \mathbb{G}$, you just have to check that $x^q = 1$. In this group, one can verify that

$$\forall x \in \mathbb{G}, \forall n \in \mathbb{Z}, x^n \pmod p = x^{n \pmod{q-1}} \pmod p.$$

   That implies that if $n \in \{1, \ldots, q\}$, then $x^{-n} \pmod p = x^{q-n} \pmod p$.
   The purpose of this exercise is to verify the last equality for $q = 5$ and $p = 2 \times 5 + 1$. First, use the method `exp` to find an element of $g \in \mathbb{Z}_p$ such that $g^q \pmod p = 1$. This ensures that $g \in \mathbb{G}$. Then check with `exp` if you have:

$$\forall n \in \{1, \ldots, q\}, g^{-n} \pmod p = g^{q-n} \pmod p.$$

   Remark: If you try to compute this equality with $x \in \mathbb{Z}_p$ such that $x^q \neq 1$, you will notice that it does not hold. This is due to the fact that in this case, $x \notin \mathbb{G}$.

Question 4: Diffie-Hellman

The goal of this question is to show how two people can establish a shared secret by exchanging only public information. We have $p = 2kq + 1$ where p and q are primes. We have the group $\mathbb{G} \subseteq \mathbb{Z}_p^*$ where $|\mathbb{G}| = q$. We choose a public generator $g \in \mathbb{G}$. Alice chooses a private secret x, computes $A = g^x \mod p$, and sends A to Bob. Bob chooses a private secret y, computes $B = g^y \mod p$, and sends B to Alice. A and B are public. Now Alice and Bob can compute the common secret K: $K = B^x \mod p$ for Alice and $K = A^y \mod p$ for Bob. In this exercise we want to show you that $K = B^x \mod p = A^y \mod p = (g^x)^y \mod p = (g^y)^x \mod p$. Compute by hand the common secret K with the following values: $p = 11$, $q = 5$, $g = 3$, $x = 6$, and $y = 7$. Calculate it from Alice's perspective and from Bob's perspective. Observe that the result is the same regardless of the perspective.